PBFT (Practical Byzantine Fault Tolerance)

📋 개요

PBFT는 1999년 Castro와 Liskov가 제안한 **최초의 실용적인 Byzantine Fault Tolerance** 알고리즘입니다. 비동기 네트워크 환경에서 악의적인 노드(Byzantine 노드)가 존재해도 합의를 달성할 수 있습니다.

핵심 특징

- **Primary-Backup 아키텍처**: 리더(Primary) 기반 합의
- **3-Phase 프로토콜**: Pre-prepare → Prepare → Commit
- **2f+1 Quorum**: n = 3f + 1 노드로 f개 Byzantine 노드 허용
- **$O(n^2)$ 메시지 복잡도**: 모든 노드가 모든 노드에게 메시지 전송
- **View Change**: Primary 실패 시 새로운 리더 선출

🎯 핵심 개념

노드 역할

- **Primary (리더)**: 클라이언트 요청을 받아 합의 시작
- **Backup (복제)**: Primary의 제안에 대해 투표
- **f**: 허용 가능한 Byzantine 노드 수
- **n = 3f + 1**: 전체 노드 수

Quorum 계산

```
n = 3f + 1
Quorum = 2f + 1

예시 (n=4):
```

```
f = 1 (1개 Byzantine 노드 허용)
Quorum = 2*1 + 1 = 3 (3개 노드 동의 필요)
```

안전성 보장

- **최소 2f+1 노드 동의**: 최소 f+1개 정직 노드 포함
- **Byzantine 노드 f개**: 나머지 f+1개는 정직하므로 올바른 결정

🔄 동작 과정

전체 흐름도

```
Client                  Primary                Backup1                Backup2
  |                       |                      |                      |
  |--REQUEST------------>|                      |                      |
  |                       |                      |                      |
  |                       |--PRE-PREPARE------->|                      |
  |                       |--PRE-PREPARE------->|-------------------->|
  |                       |--PRE-PREPARE------->|-------------------->|------------
  |                       |                      |                      |
  |                       |<----PREPARE---------|                      |
  |                       |<----PREPARE---------|<--------------------|
  |                       |<----PREPARE---------|<--------------------|<-----------
  |                       |-----PREPARE-------->|                      |
  |                       |-----PREPARE-------->|-------------------->|
  |                       |-----PREPARE-------->|-------------------->|------------
  |                       |                      |                      |
  |                       |                      |<---PREPARE---------|
  |                       |                      |<---PREPARE---------|<-----------
  |                       |                      |----PREPARE-------->|
  |                       |                      |----PREPARE-------->|------------
  |                       |                      |                      |
  |                       |<----COMMIT----------|                      |
  |                       |<----COMMIT----------|<--------------------|
  |                       |<----COMMIT----------|<--------------------|<-----------
  |                       |-----COMMIT--------->|                      |
  |                       |-----COMMIT--------->|-------------------->|
  |                       |-----COMMIT--------->|-------------------->|------------
  |                       |                      |                      |
  |<--REPLY-------------|                      |                      |
```

```
|<--REPLY-------------|---------------------|                    |
|<--REPLY-------------|---------------------|--------------------|------------
```

Phase 1: Pre-prepare (Primary → All Backups)

목적: Primary가 클라이언트 요청을 받아 제안

동작:

1. Client가 Primary에게 요청 전송
2. Primary가 요청에 sequence number 할당
3. Primary가 모든 Backup에게 PRE-PREPARE 메시지 전송

메시지 구조:

```
PRE-PREPARE {
    view:     현재 view 번호
    sequence: 요청 순서 번호
    digest:   요청의 해시값
    request:  원본 요청 데이터
}
```

검증 조건 (Backup이 확인):

- 올바른 view 번호
- 동일 view/sequence에 다른 digest 없음
- sequence 번호가 범위 내 (watermark)
- 서명 유효성

Phase 2: Prepare (All Nodes → All Nodes)

목적: 모든 노드가 같은 순서로 요청을 받았음을 확인

동작:

1. PRE-PREPARE를 받은 Backup이 검증
2. 검증 통과 시 모든 노드에게 PREPARE 메시지 전송
3. 각 노드가 2f개의 PREPARE 수신 대기 (자신 포함 2f+1)

메시지 구조:

```
PREPARE {
    view:     현재 view 번호
    sequence: 요청 순서 번호
    digest:   요청의 해시값
    nodeID:   발신 노드 ID
}
```

Prepared 상태:
노드가 다음을 받으면 “prepared”:

- 1개의 PRE-PREPARE(v, n, d)
- 2f개의 일치하는 PREPARE(v, n, d) (다른 노드로부터)

의미: 2f+1개 노드가 동일한 요청을 동일한 순서로 받았음을 증명

Phase 3: Commit (All Nodes → All Nodes)

목적: 모든 정직한 노드가 같은 순서로 실행하도록 보장

동작:

1. Prepared 상태 도달한 노드가 COMMIT 메시지 전송
2. 각 노드가 2f+1개의 COMMIT 수신 대기
3. 2f+1 COMMIT 받으면 요청 실행

메시지 구조:

```
COMMIT {
    view:     현재 view 번호
    sequence: 요청 순서 번호
    digest:   요청의 해시값
    nodeID:   발신 노드 ID
}
```

Committed 상태:
노드가 다음을 받으면 “committed”:

- Prepared 상태
- 2f+1개의 일치하는 COMMIT(v, n, d)

실행 및 응답:

1. 요청 실행 (상태 머신 업데이트)
2. Client에게 REPLY 전송
3. Client는 f+1개의 동일한 REPLY 대기

📊 메시지 흐름 상세 분석

4노드 예제 (n=4, f=1)

초기 상태:

- Node 0: Primary
- Node 1, 2, 3: Backups
- Quorum = 3

단계별 메시지 수:

1. **Pre-prepare**: 1개 (Primary → 3 Backups) = 3 messages
2. **Prepare**: 4개 노드 × 3개 타겟 = 12 messages
3. **Commit**: 4개 노드 × 3개 타겟 = 12 messages
4. **Reply**: 4개 노드 → Client = 4 messages

총 메시지: 3 + 12 + 12 + 4 = **31 messages per request**

메시지/트랜잭션: 약 **8-10 messages** (배치 처리 시)

타임라인

```
t=0:    Client → Primary (REQUEST)
t=1:    Primary → All (PRE-PREPARE)
t=2:    All → All (PREPARE) - 12 messages
t=3:    Prepared 상태 도달
t=4:    All → All (COMMIT) - 12 messages
t=5:    Committed 상태 도달, 실행
t=6:    All → Client (REPLY)
```

🔍 안전성 및 활성 증명

안전성 (Safety)

정리: 정직한 두 노드가 같은 sequence에 대해 다른 요청을 실행하지 않음

증명 스케치:

1. 노드가 요청 실행 = 2f+1 COMMIT 받음
2. 2f+1 COMMIT = 최소 f+1 정직 노드 포함
3. 정직 노드는 하나의 요청에만 PREPARE 전송
4. 따라서 다른 요청이 2f+1 COMMIT을 받을 수 없음

활성 (Liveness)

전제 조건:

- 최대 f개 Byzantine 노드
- 네트워크가 결국 메시지 전달 (비동기)
- Primary가 정직하거나 View Change 발생

보장:

- 정직한 Primary: 요청이 결국 실행됨
- Byzantine Primary: View Change로 새 Primary 선출

🔄 View Change

발생 조건

- Primary가 타임아웃 내 응답 없음
- Primary가 잘못된 메시지 전송
- Backup이 Primary 의심

View Change 프로세스

```
Node                    All Nodes
  |                        |
  |---VIEW-CHANGE-------->|  (timeout)
  |                        |
```

```
|                        |  수집 2f+1 VIEW-CHANGE
|                        |
|<--NEW-VIEW------------|  (new Primary)
|                        |
```

1. **VIEW-CHANGE 메시지**:

```
VIEW-CHANGE {
    newView:  v + 1
    nodeID:   발신자
    prepared: prepared 상태 증명
}
```

2. **새 Primary 선출**:
    - Primary = view mod n
    - Example: view 1, n=4 → Primary = Node 1

3. **NEW-VIEW 메시지**:

```
NEW-VIEW {
    newView:     v + 1
    viewChanges: 2f+1개의 VIEW-CHANGE
    preprepares: 재시작할 요청들
}
```

💻 구현 세부사항

핵심 데이터 구조

```go
type PBFTNode struct {
    id           int
    view         int           // 현재 view
    sequence     int           // 마지막 sequence number

    prepareVotes map[int]map[int]bool  // [seq][nodeID]
    commitVotes  map[int]map[int]bool  // [seq][nodeID]

    prepared     map[int]bool   // prepared 상태
    committed    map[int]bool   // committed 상태
}
```

주요 함수

메시지 처리:

- `handleRequest()` : Client 요청 처리 (Primary만)
- `handlePrePrepare()` : PRE-PREPARE 검증 및 PREPARE 전송
- `handlePrepare()` : PREPARE 수집, 2f+1 도달 시 COMMIT
- `handleCommit()` : COMMIT 수집, 2f+1 도달 시 실행

검증 로직:

```go
func (node *PBFTNode) handlePrepare(msg Message) {
    seq := msg.Sequence
    senderID := msg.SenderID

    // 투표 기록
    if node.prepareVotes[seq] == nil {
        node.prepareVotes[seq] = make(map[int]bool)
    }
    node.prepareVotes[seq][senderID] = true

    // Quorum 확인 (2f+1)
    if len(node.prepareVotes[seq]) >= node.quorum {
        node.prepared[seq] = true
        node.broadcast(COMMIT, seq)
    }
}
```

코드 위치

- 구현: `internal/algorithms/pbft/pbft.go`
- 테스트: `internal/algorithms/pbft/pbft_test.go`
- 메시지 정의: `internal/common/message.go`

📈 성능 특성

장점

✅ 이론적으로 증명된 안전성
✅ 비동기 네트워크 지원
✅ View change로 활성 보장
✅ 실제 사용 가능 (Hyperledger Fabric 등)

단점

❌ $O(n^2)$ 메시지 복잡도 → 확장성 제한
❌ 3단계 프로토콜 → 높은 지연시간
❌ 모든 노드 간 통신 → 네트워크 부하

최적화 기법

- **Batching**: 여러 요청을 하나의 블록으로 처리
- **MAC 인증**: 디지털 서명 대신 경량 인증
- **Checkpointing**: 주기적 상태 스냅샷
- **Tentative execution**: Prepare 후 투기적 실행

🔬 벤치마크 결과

우리 구현 (4노드, 100트랜잭션):

- **TPS**: 1.0
- **메시지/트랜잭션**: 8.1
- **지연시간**: ~10ms (시뮬레이션)

🎓 학습 포인트

1. **2f+1 Quorum의 의미**:
    - 최소 f+1개 정직 노드를 보장
    - Byzantine 노드가 결과에 영향 못 줌
2. **3-Phase의 필요성**:
    - Pre-prepare: 순서 제안

    - Prepare: 순서 합의
    - Commit: 실행 보장

3. **메시지 복잡도**:

    - 모든 노드가 모든 노드에게 전송 = $O(n^2)$
    - 이것이 PBFT의 주요 확장성 한계

📚 참고 자료

- [원본 논문] Castro, M., & Liskov, B. (1999). “Practical Byzantine Fault Tolerance”
- [코드 구현] `internal/algorithms/pbft/pbft.go`
- [HotStuff와 비교] PBFT는 $O(n^2)$, HotStuff는 $O(n)$

다음: Tendermint →